Name of the Student	Shaik Mubarak
Reg. No	192425194
GitHub Link	https://github.com/mrclassic7499/MLA0405-DEEP-LEARNING/tree/main/Assessment/CO2

SIMATS ENGINEERING

CO2 - ASSESSMENT TOOL 2

Scenario-Based Questions

COURSE NAME: Deep Learning **COURSE CODE:MLA0405**
Faculty : Dr. Arul Raj A.M **Slot : D**

COURSE OUTCOME COVERED
CO 2: Students will be able to apply supervised learning algorithms such as linear regression, classification models, decision trees, neural networks, support vector machines, and ensemble methods to solve real-world prediction and classification problems. They will gain the ability to select appropriate models, train them using datasets, and evaluate their performance. Students will also understand the strengths and limitations of different supervised learning approaches.(BTL-4)
CO Weightage : 15%

Assessment Tool Weightage: 40%

Duration: 90 minutes

Marks: 25

Code of Conduct:

I Shaik Mubarak(192425194) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **Name of the student:Shaik Mubarak**

Scenario-Based Questions

CO-AT2

QUESTIONS: 05 **Total Marks:25**

1. **Unsupervised Training of Neural Networks, Auto Encoders**
   A hospital wants to identify abnormal ECG patterns from thousands of unlabeled records. Which deep learning technique from the syllabus would you recommend? Justify your choice and explain the workflow.

Answer:

AUTOENCODER (UNSUPERVISED NEURAL NETWORK)

Concept Identification

The hospital has a large collection of unlabeled ECG (Electrocardiogram) signals, making supervised learning unsuitable because there are no labels indicating whether a signal is normal or abnormal.An Autoencoder, an unsupervised deep learning model, is the most suitable technique. It learns the normal characteristics of ECG signals by compressing them into a lower-dimensional representation and then reconstructing them. Abnormal ECG signals generally produce a higher reconstruction error, allowing them to be detected as anomalies.

Justification

The Autoencoder is recommended because:

- It does not require labeled training data.
- It automatically learns meaningful features from ECG signals.
- It detects anomalies using reconstruction error.
- It reduces the need for manual feature engineering.
- It is widely used in healthcare for medical signal analysis and anomaly detection.

Workflow

Step 1: Data Collection

- Collect thousands of unlabeled ECG recordings from patients.

↓

Step 2: Data Preprocessing

- Remove noise from ECG signals.
- Normalize signal values.
- Convert signals into a suitable numerical format.

↓

Step 3: Train Autoencoder

- Input ECG signals into the encoder.
- Encoder compresses the data into a latent representation.
- Decoder reconstructs the original ECG signal.
- The model minimizes reconstruction loss during training.

↓

Step 4: Reconstruction Error Calculation

- Compare original ECG with reconstructed ECG.
- Calculate Mean Squared Error (MSE).

↓

Step 5: Abnormality Detection

- Low reconstruction error → Normal ECG.
- High reconstruction error → Abnormal ECG.

↓

Step 6: Medical Review

- Flag abnormal ECGs for cardiologists to examine.

Architecture Diagram

ECG Signal

▼

Data Preprocessing

▼

Encoder

Latent Representation

▼

Decoder

▼

Reconstructed ECG

▼

Reconstruction Error

Low Error
Normal

High Error
Abnormal

Advantages

- No labeled data required.
- Learns hidden ECG features automatically.
- Detects rare and unknown abnormalities.
- Reduces manual analysis effort.
- Suitable for large healthcare datasets.

Limitations

- Requires a sufficient amount of normal ECG data for effective training.
- Performance depends on selecting an appropriate anomaly threshold.
- Extremely noisy data can reduce detection accuracy.

Integration of Literature

Autoencoders are widely used in medical image and biosignal analysis for anomaly detection. Research has shown that deep autoencoders effectively detect abnormal ECG patterns by learning compact representations of normal cardiac activity and identifying signals with high reconstruction error. Similar approaches are commonly used in healthcare monitoring, fault detection, and predictive maintenance.

Conclusion

An Autoencoder is the most appropriate deep learning technique for detecting abnormal ECG patterns from unlabeled records. By learning the characteristics of normal ECG signals and identifying high reconstruction errors, it enables efficient and accurate anomaly detection without requiring labeled data, making it highly suitable for hospital applications.

---

2. **Auto Encoders, Representation Learning**
   A retail company wants to reduce the dimensionality of customer purchase data before performing customer segmentation. Explain how an autoencoder can be used to solve this problem.

Answer:

AUTOENCODER FOR REPRESENTATION LEARNING AND DIMENSIONALITY REDUCTION

Concept Identification

Customer purchase datasets often contain hundreds or thousands of features, such as:

- Purchase frequency
- Product categories
- Spending amount
- Payment methods
- Shopping time
- Customer demographics
- Online browsing history

High-dimensional data increases computational complexity and may contain redundant information. An Autoencoder is an unsupervised neural network that learns a compact (low-dimensional) representation of the original data while preserving its important characteristics. This compressed representation can then be used for customer segmentation.

Justification

An Autoencoder is suitable because it:

- Learns meaningful customer behavior patterns automatically.
- Removes redundant and less useful features.

- Produces a compact latent representation.
- Improves the efficiency and accuracy of clustering algorithms.
- Does not require labeled customer data.

Workflow

Step 1: Data Collection

- Gather customer purchase history from online and offline transactions.

↓

Step 2: Data Preprocessing

- Handle missing values.
- Normalize numerical features.
- Encode categorical variables.

↓

Step 3: Train the Autoencoder

- Input customer purchase data into the encoder.
- Encoder compresses the data into a low-dimensional latent space.
- Decoder reconstructs the original purchase data.
- Minimize reconstruction loss during training.

↓

Step 4: Extract Latent Features

- Use the encoder output as compact customer representations.

↓

Step 5: Customer Segmentation

- Apply clustering algorithms (e.g., K-Means) on the latent features.
- Group customers with similar purchasing behavior.

↓

Step 6: Business Decision

- Use customer segments for personalized marketing, product recommendations, and promotional offers.

Architecture Diagram

```
      Customer Purchase Data
                │
                ▼
        Data Preprocessing
                │
                ▼
        ┌───────────────────┐
        │      Encoder      │
        └───────────────────┘
                │
    Low-Dimensional Features
         (Latent Space)
                │
                ▼
        ┌───────────────────┐
        │      Decoder      │
        └───────────────────┘
                │
                ▼
   Reconstructed Purchase Data
                │
                ▼
   Latent Features → K-Means
                │
                ▼
        Customer Segments
```

Linkages Between Concepts

- High-dimensional purchase data contains redundant information.

- Autoencoder compresses the data into meaningful latent features.
- Representation Learning captures hidden customer behavior patterns.
- Customer Segmentation groups customers based on these learned features.
- Businesses use these groups for targeted marketing and personalized recommendations.

Advantages

- Reduces data dimensionality effectively.
- Learns important customer behavior automatically.
- Removes redundant information.
- Improves clustering performance.
- Faster processing with lower computational cost.
- Supports personalized marketing strategies.

Limitations

- Requires sufficient training data.
- Training deep autoencoders can be computationally intensive.
- If the latent dimension is too small, important information may be lost.

Integration of Literature

Autoencoders are widely used in retail analytics for representation learning and dimensionality reduction. Research shows that compressed latent features generated by autoencoders often improve clustering quality compared to using raw high-dimensional data. Many e-commerce platforms use these techniques for customer segmentation, recommendation systems, and personalized marketing.

Real-World Applications

- Amazon – Product recommendation systems.
- Flipkart – Customer purchase analysis.
- Walmart – Market basket analysis.
- Netflix – User preference representation.
- E-commerce platforms – Personalized promotions and targeted advertisements.

Conclusion

An Autoencoder is an effective solution for reducing the dimensionality of customer purchase data. By learning compact representations of customer behavior, it simplifies the dataset while preserving important information. These latent features improve customer segmentation, enabling businesses to deliver personalized marketing strategies, better recommendations, and enhanced customer experiences.

3. **Width and Depth of Neural Networks, Activation Functions**
   A facial recognition system performs poorly when trained using a shallow neural network. Recommend architectural changes involving network depth and activation functions.

Answer:

INCREASE THE NETWORK DEPTH AND USE RELU-BASED ACTIVATION FUNCTIONS

Concept Identification

A shallow neural network contains only one or two hidden layers and has limited ability to learn complex facial features. Facial recognition involves recognizing intricate patterns such as facial shape, eyes, nose, mouth, skin texture, and expressions. Therefore, a deep neural network (DNN) with multiple hidden layers and suitable activation functions is recommended.The ReLU (Rectified Linear Unit) activation function is widely used because it speeds up training, reduces the vanishing gradient problem, and enables the network to learn complex non-linear relationships.

Justification

The facial recognition system performs poorly because a shallow network cannot extract high-level facial features. Increasing the network depth allows the model to learn hierarchical representations:

- Early layers detect edges and corners.
- Middle layers identify facial components such as eyes, nose, and mouth.
- Deep layers recognize complete facial structures and identities.

Using ReLU in hidden layers improves convergence and learning efficiency, while Softmax in the output layer provides class probabilities for facial recognition.

Recommended Architectural Changes

1. Increase Network Depth

- Replace the shallow network with a deep neural network.
- Use multiple hidden layers (4–10 or more depending on the dataset).
- Each layer learns progressively more complex facial features.

2. Use ReLU Activation

- Hidden Layers → **ReLU**
- Benefits:
  - Faster convergence
  - Prevents vanishing gradients
  - Efficient computation

3. Use Softmax Output Layer

- Produces the probability for each person's identity.
- Selects the class with the highest probability.

4. Add Regularization

- **Dropout** reduces overfitting.
- **Batch Normalization** stabilizes and accelerates training.

Workflow

Step 1: Collect Facial Images

- Acquire labeled facial images from different individuals.

↓

Step 2: Data Preprocessing

- Resize images.
- Normalize pixel values.
- Augment data (rotation, flipping, brightness adjustment).

↓

Step 3: Build Deep Neural Network

- Input Layer
- Multiple Hidden Layers with ReLU
- Batch Normalization
- Dropout Layers
- Output Layer with Softmax

↓

Step 4: Model Training

- Train using backpropagation and gradient descent.
- Minimize classification loss.

↓

Step 5: Prediction

- The network identifies the person's face with the highest confidence score.

Architecture Diagram

Facial Image
↓
Data Preprocessing
↓
Input Layer
↓
Hidden Layer 1 (ReLU)
↓
Hidden Layer 2 (ReLU)
↓
Hidden Layer 3 (ReLU)
↓
Batch Normalization
↓
Dropout Layer

↓

Output Layer (Softmax)

↓

Predicted Person ID

Linkages Between Concepts

- Deep Neural Network → Learns hierarchical facial features.
- ReLU Activation → Improves learning speed and avoids vanishing gradients.
- Batch Normalization → Stabilizes training.
- Dropout → Prevents overfitting.
- Softmax → Classifies the face into the correct identity.

Advantages

- Higher facial recognition accuracy.
- Learns complex facial features effectively.
- Faster convergence using ReLU.
- Better generalization through Dropout.
- Improved training stability with Batch Normalization.

Limitations

- Requires more computational resources.
- Longer training time.
- Needs a larger labeled dataset.
- Risk of overfitting if regularization is not applied.

Integration of Literature

Deep neural networks with ReLU activation have become the standard approach for facial recognition due to their superior feature extraction capabilities. Modern systems such as DeepFace, FaceNet, and VGGFace use deep architectures to learn hierarchical facial representations, achieving significantly higher recognition accuracy than shallow neural networks.

Real-World Applications

- Smartphone Face Unlock
- Airport Biometric Security
- Attendance Management Systems
- Banking Identity Verification
- CCTV Surveillance Systems

Conclusion

A deep neural network with multiple hidden layers and ReLU activation functions is the recommended solution for improving facial recognition performance. The increased depth enables hierarchical feature learning, while ReLU accelerates training and addresses the vanishing gradient problem. Incorporating Batch Normalization, Dropout, and a Softmax output layer further improves accuracy, stability, and generalization, making the system suitable for real-world facial recognition applications.

---

4. **Restricted Boltzmann Machines (RBMs), Unsupervised Training of Neural Networks**
   A streaming service wants to recommend movies to users based on their viewing history without using labeled data. Explain how RBMs can be applied to build the recommendation engine.

Answer:

RESTRICTED BOLTZMANN MACHINE (RBM) FOR UNSUPERVISED RECOMMENDATION SYSTEMS

Concept Identification

A Restricted Boltzmann Machine (RBM) is an unsupervised neural network that learns hidden patterns and relationships in user viewing behavior without requiring labeled data. It consists of two layers:

- **Visible Layer** – Represents users' movie ratings or viewing history.
- **Hidden Layer** – Learns hidden preferences and interests of users.

By discovering these hidden patterns, the RBM predicts movies that a user is likely to watch, making it suitable for recommendation systems.

Justification

RBMs are recommended because they:

- Learn from unlabeled viewing history.
- Capture hidden relationships between users and movies.
- Predict user preferences based on learned patterns.
- Handle sparse user–movie interaction data effectively.
- Generate personalized movie recommendations.

Workflow

Step 1: Data Collection

- Collect users' movie viewing history and ratings.

↓

Step 2: Data Preprocessing

- Create a User–Movie Interaction Matrix.
- Normalize ratings or convert watched/not watched into binary values.

↓

Step 3: Train the RBM

- Feed the interaction matrix into the visible layer.
- Hidden layer learns latent features such as preferred genres, actors, or viewing habits.
- Update weights using Contrastive Divergence (CD).

↓

Step 4: Learn User Preferences

- Hidden neurons capture users' interests automatically.

↓

Step 5: Predict Missing Ratings

- Estimate ratings for movies the user has not watched.

↓

Step 6: Generate Recommendations

- Recommend movies with the highest predicted ratings.

Architecture Diagram

```
        User-Movie Matrix
               │
               ▼
       Data Preprocessing
               │
               ▼
┌─────────────────────────────┐
│ Visible Layer               │
│ (Movies Watched)            │
└─────────────────────────────┘
               │
        Learned Weights
               │
               ▼
┌─────────────────────────────┐
│ Hidden Layer                │
│ User Preferences            │
└─────────────────────────────┘
               │
               ▼
     Predicted Movie Ratings
               │
               ▼
  Personalized Recommendations
```

Linkages Between Concepts

- Viewing History provides user interaction data.
- RBM learns hidden user preferences without labels.
- Hidden Layer extracts latent features from viewing patterns.
- Predicted Ratings estimate interest in unseen movies.
- Recommendation Engine suggests personalized content.

Advantages

- Does not require labeled data.
- Learns hidden user preferences automatically.
- Handles large-scale recommendation problems.
- Improves personalization.
- Reduces manual feature engineering.

Limitations

- Training can be computationally expensive.
- Performance depends on the amount of user interaction data.
- New users or new movies (cold-start problem) may reduce recommendation quality.
- Modern deep learning models may outperform RBMs on very large datasets.

Integration of Literature

Restricted Boltzmann Machines became popular in recommendation systems after their successful application to collaborative filtering problems. Research has shown that RBMs effectively learn latent user preferences from viewing histories and rating data. They have been used in movie recommendation tasks and influenced the development of modern recommender systems.

Real-World Applications

- Netflix – Personalized movie recommendations.
- Amazon Prime Video – Movie and TV show suggestions.
- Disney+ – Content recommendation.
- Spotify – Music recommendation.
- YouTube – Personalized video suggestions.

Conclusion

A Restricted Boltzmann Machine (RBM) is an effective unsupervised learning model for movie recommendation systems. By learning hidden patterns from users' viewing history, it predicts preferences for unseen movies and generates personalized recommendations. This approach improves user engagement and satisfaction without requiring labeled training data.

---

5. **Deep Learning Applications, Representation Learning**

   A manufacturing company wants to detect defective products from sensor readings collected every second. Design a deep learning solution using concepts from representation learning and neural networks.

Answer:

AUTOENCODER WITH DEEP NEURAL NETWORK (DNN)

Concept Identification

In a manufacturing environment, sensors continuously collect data such as temperature, pressure, vibration, humidity, and machine speed. Since defective products are rare and labeled data may be limited, representation learning using an Autoencoder is an effective approach.The Autoencoder learns compact representations (latent features) of normal sensor data. These learned features are then used by a Deep Neural Network (DNN) to classify products as Normal or Defective.

Justification

The proposed solution is suitable because:

- It automatically learns important patterns from high-dimensional sensor data.
- It reduces noise and redundant information.
- It improves defect detection accuracy.
- It supports real-time monitoring of manufacturing processes.
- It minimizes manual feature engineering.

Workflow

Step 1: Sensor Data Collection

- Collect real-time sensor readings every second.
- Example parameters:
  - Temperature
  - Pressure
  - Vibration
  - Humidity
  - Machine Speed

↓

Step 2: Data Preprocessing

- Remove missing values.
- Normalize sensor readings.
- Filter noisy data.

↓

Step 3: Representation Learning

- Train an Autoencoder using sensor data.
- Encoder compresses the input into latent features.
- Decoder reconstructs the original data.
- The latent representation captures the most important characteristics.

↓

Step 4: Neural Network Classification

- Feed the latent features into a Deep Neural Network (DNN).
- The DNN learns to classify products as:
  - Normal
  - Defective

↓

Step 5: Defect Detection

- If the model predicts Defective, send an alert to the quality control team.
- Remove defective products from the production line.

Architecture Diagram

```
             Sensor Readings
  (Temperature, Pressure, Vibration)
                    │
                    ▼
           Data Preprocessing
                    │
                    ▼
        ┌─────────────────────┐
        │    │  Encoder   │   │
        └─────────────────────┘
                    │
             Latent Features
                    │
                    ▼
```

Deep Neural Network

↓

Normal / Defective

↓

Quality Control Alert

Linkages Between Concepts

- Sensor Data provides continuous machine information.
- Representation Learning (Autoencoder) extracts meaningful latent features.
- Deep Neural Network classifies products using these features.
- Classification Results help identify defective products.
- Quality Control System removes faulty products and improves manufacturing efficiency.

Advantages

- Learns meaningful representations automatically.
- Detects defects with high accuracy.
- Handles high-dimensional sensor data.
- Supports real-time monitoring.
- Reduces production losses and maintenance costs.

Limitations

- Requires sufficient training data.
- Deep models require higher computational resources.
- Sensor failures or noisy data can reduce prediction accuracy.
- Periodic retraining may be needed as production conditions change.

Integration of Literature

Representation learning with Autoencoders and Deep Neural Networks is widely used in Industry 4.0 for predictive maintenance, fault diagnosis, and quality inspection. Research demonstrates that learning latent representations from sensor data improves defect detection accuracy compared with manual feature engineering.

Real-World Applications

- Automobile manufacturing – Detect engine and component defects.
- Electronics industry – Identify faulty circuit boards.
- Food processing – Monitor product quality.
- Pharmaceutical industry – Detect packaging defects.
- Smart factories (Industry 4.0) – Real-time quality monitoring.

Conclusion

A combination of representation learning using an Autoencoder and a Deep Neural Network provides an effective solution for detecting defective products from continuous sensor readings. The Autoencoder extracts meaningful latent features from complex sensor data, while the DNN accurately classifies products as normal or defective. This approach enhances product quality, reduces waste, and supports efficient real-time manufacturing.

RUBRICS FOR CALCULATION:

Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established
Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand